UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
Universidad del Perú, Decana de América
FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA
ESCUELA PROFESIONAL DE INGENIERÍA DE SOFTWARE

TEST PLAN, TEST DESIGN SPEC, TEST CASE SPEC

Y TEST PROCEDURE SPEC

INTEGRANTES :

- Cano Vasquez, Juan José
- Munayco Sirlopu, Ronald Alfredo
- Osorio Montenegro, Sergio Alejandro
- Sandoval Falcon, David Brian
- Villanueva Aguirre, Cesar Alexander

ASIGNATURA:

MÉTODOS FORMALES PARA PRUEBAS

DOCENTE:

Arturo Alejandro Bartra More

Lima, Perú, 2025

TEST PLAN

1. Identificador del Test plan

SW-TP01-25

2. Introducción

2.1. Objetivos

Los objetivos con los documentos de pruebas para el presente proyecto de e-commerce son los siguientes:

(1) Desarrollar un plan de pruebas de calidad en base a la norma IEEE 829-1998
(2) Establecer un marco de pruebas organizado
(3) Comunicar todos los elementos necesarios para el adecuado desarrollo de las pruebas

2.2. Antecedentes

La empresa Great Fit S.A, empresa dedicada al ofrecimiento de productos y servicios relacionados al bienestar físico y fitness, solicitó una solución de software con el fin de propagar su negocio en el entorno digital y poder ofrecer sus productos a través de envíos.

Se presentó un documento especificando las funcionalidades que se desarrollarían para esta solución y se procedió a su implementación el 27 de julio del 2025. Con parte en los lineamientos de la empresa se continuó con la etapa de pruebas.

2.3. Alcance

El presente Test Plan cubre todas las funcionalidades desarrolladas para la aplicación web de e-commerce para Great Fit que alcanzaron a ser desarrolladas hasta la entrega el 27 de julio del 2025

2.4. Referencias

Los siguientes documentos fueron utilizados como recursos para el test plan:

Perfil del proyecto para Great Fit (SW-PP01-25)
Documentación de requisitos para aplicación web de e-commerce Great Fit (SW-DR01-25)
Documento de análisis de requerimientos para aplicación web de e-commerce Great Fit (SW-DA01-25)

3. Test Items

Los documentos usados en esta sección son:

Perfil del proyecto para Great Fit (SW-PP01-25)
Documentación de requisitos para aplicación web de e-commerce Great Fit (SW-DR01-25)
Documento de análisis de requerimientos para aplicación web de e-commerce Great Fit (SW-DA01-25)

3.1. Módulos de programa

Los ítems a probar son los siguientes:

Tipo	Ejecución	Nombre de miembro
Interfaz Frontend	npm run dev	FW - Front
Interfaz Backend	npm run server	FW - Back
Interfaz Admin	npm run dev	FW - Admin

4. Características a ser probadas

La siguiente lista describe las características a probar:

ID característica	Descripción	Test Design asociado
AP06-01	Registro y gestión de usuarios	SW-TD01.01-25
AP06-02	Selección de productos del catálogo y carrito	SW-TD01.01-25
AP06-03	Realizar pagos en línea	SW-TD01.01-25
AP06-04	Registro y modificación de productos	SW-TD01.01-25
AP06-05	Actualización de estado del pedido	SW-TD01.01-25
AP06-06	Generación de planes de entrenamiento con IA	SW-TD01.01-25

5. Características a no ser probadas

Funcionalidades no implementadas al 27 de julio del 2025

6. Enfoque para las pruebas

El equipo de pruebas usará la documentación del sistema para desarrollar la documentación de las pruebas, incluyendo el Test Design, Test Case y Test Procedures. Con este enfoque se busca verificar el funcionamiento adecuado del sistema y correctitud de la documentación.

Pruebas de caja negra

6.1. Flujo de casos de uso

Realizar los pasos asociados al uso del sistema y comprobar que se cumplan correctamente

6.2. Análisis de valores límite

Para comprobar que las entradas de los campos en los formularios admitan valores consistentes a su propósito

6.3. Tablas de decisión

Para comprobar que las entradas resultan mostrando valores correctos a lo largo de la aplicación y evaluar el cumplimiento de flujos lógicos

<u>Pruebas de caja blanca</u>

6.4. Técnica de Cobertura de sentencias

Realizar los pasos asociados al uso del sistema y comprobar que se cumplan correctamente

6.5. Técnica de Cobertura de decisión

Para probar los posibles resultados de un flujo lógico

7. Criterios de fallo y éxito

Para que un ítem pase la prueba se debe documentar que cumplió con el requerimiento asociado de acuerdo a la técnica utilizada y haciendo uso de métricas

8. Criterios de suspensión y reanudación

8.1. De suspensión

Si se encuentra un defecto que impide continuar con las pruebas

8.2. De reanudación

Cuando el defecto que impedía continuar con las pruebas es corregido

9. Entregables de pruebas

9.1. Plan de Pruebas (Test Plan)
9.2. Especificaciones de Diseño de Prueba (Test Design)
9.3. Especificaciones de Casos de Prueba (Test Cases)
9.4. Especificaciones de Procedimiento de Prueba (Test Procedures)

10. Tareas de testing

Tarea	Pre-requisito	Responsabilidad	Grado de esfuerzo
(1) Preparar el test plan		El equipo de pruebas	4
(2) Preparar el Test Design Specification	Tarea 1	El equipo de pruebas	4
(3) Preparar el Test	Tarea 2	El equipo de pruebas	4

procedure Specification			
(3) Prepara el sistema para las pruebas	Tarea 3	El equipo de pruebas	4
(4) Ejecutar cada prueba por categoría	Tarea 4	El equipo de pruebas	3
(5) Realizar el reporte de las pruebas	Tarea 5	El equipo de pruebas	2
(6) Documentar las pruebas	Tarea 3	El equipo de pruebas	1

11. Necesidades de entorno

11.1. Hardware

Cualquier PC de gama media-baja en adelante

11.2. Software

Windows 7/10/11 y cualquier navegador

11.3. Herramientas

Node package manager y otras herramientas a determinar para las pruebas.

11.4. Publicaciones

Documentación de la aplicación web de e-commerce para Great Fit.

12. Responsabilidades

12.1. Grupo de prueba del sistema

Ejecutar y gestionar las pruebas para la aplicación web para Great Fit y revisar los documentos de pruebas.

12.2. Grupo de desarrollo del proyecto

Proporcionar la documentación y apoyo relacionados a las pruebas en caso sea necesario

13. Necesidades de capacitación y personal

El equipo de pruebas necesitará recibir apoyo para generar correctamente las pruebas pues solo se dispone de la documentación de la etapa de análisis y requerimientos

14. Cronograma

Las actividades para la etapa de pruebas inician desde el mes de Octubre del 2025 hasta el 1ro de Diciembre del 2025

15. Riesgos y contingencias

Riesgo 1: Falta de tiempo para probar las funcionalidades
Contingencia: Se priorizará los siguientes ítems de pruebas. AP06-01. AP06-02. AP06-03. AP06-04. AP06-05

16. Aprobación

El documento se aprobó el 28 de octubre del 2025

TEST DESIGN

1. Identificador del Test Design Specification

SW-TD01.01-25

2. Características a ser probadas

Documento asociado	Requisito - ID característica (Test Plan)	Item de prueba	Descripción
Documentación y especificación de requisitos	RF1 - AP06-01 RF2 - AP06-05 RF3 - AP06-02 RF4 - AP06-02 … RF5 - AP06-04 RF7 - AP06-03	IP01 IP02 IP03 IP04 IP05 IP06 IP07 IP08 IP09 IP11	Inicio de sesión Registro de usuario Modificación de estado de los pedidos Filtrar catálogo Solicitar pedido Ver historia de pedidos Ver estado del pedido Registrar producto Eliminar producto Pago de pedido

3. Detalle del enfoque

Con las características anteriormente identificadas se detalla las técnicas a utilizar por cada ítem de prueba a continuación

Item de prueba	Técnicas a utilizar	Justificación

IP01	Análisis de valores límite Tablas de decisión Técnica de cobertura de decisión	contraseña #dígitos validez de campos de usuario y contraseña Revisar los caminos lógicos del controlador loginUser (Backend)
IP02	Análisis de valores límite Tablas de decisión Técnica de cobertura de decisión	contraseña #dígitos Validez de campos de usuario y contraseña Revisar los caminos lógicos del controlador registerUser (Backend)
IP03 IP04	Transición de estados Técnica de cobertura de decisión	Cada categoría puede representar un estado Se hace uso de categorías, se trata de filtrar usando un condicional
IP05	Partición de equivalencia Técnica de cobertura de decisión	campos vacíos o llenos en el formulario de envío Revisar los caminos lógicos del controlador placeOrder (Backend)
IP06 IP07	Técnica de cobertura de sentencias Transición de estados	Verificar que el flujo normal y el flujo de error se apliquen correctamente
IP11	Transición de estados	Existen 3 estados del pedido

4. Identificación de prueba

Test Case ID	Nombre	Item de prueba	Descripción
Caja Negra			
TC02	Selección de productos de catálogo y carrito	IP04 e IP05	Verificar correcta funcionalidad del carrito y filtrado de catálogo
TC01	Campo contraseña	IP01	Verificar que el campo tenga entradas válidas
TC01	Login correcto	IP01	Campos de usuario y contraseña existen y son válidos o no válidos
TC01	Campo contraseña	IP02	Verificar que el campo tenga el mínimo requerido
TC01	Registro correcto	IP02	Campos de usuario, correo y contraseña son validos

TC03	Campos de registro de producto válido	IP08	Los campos están bien consistenciados y no permiten agregar valores sin sentido
TC03	Eliminación correcta del producto	IP09	La eliminación solo afecta al producto y desaparece de la BD
Caja Blanca			
TC04	Cobertura de decisión de registerUser	IP02	Realizar las pruebas necesarias para cubrir las decisiones relacionadas al controlador registerUser
TC05	Cobertura de decisión de placeOrder	IP05	Realizar las pruebas necesarias para cubrir las decisiones relacionadas al controlador placeOrder
TC06	Cobertura de decisión de verifyOrder	IP11	Realizar las pruebas necesarias para cubrir las decisiones relacionadas al controlador verifyOrder
TC07	Cobertura de decisión de loginUser	IP01	Realizar las pruebas necesarias para cubrir las decisiones relacionadas al controlador loginUser
TC08	Cobertura de decisión del componente Productos Display (Frontend)	IP04	Verificar el correcto funcionamiento del filtrado de catálogo
TC09	Cobertura de sentencias del controlador userOrders (Backend)	IP06	Comprobar que se responda correctamente ante errores y flujo normal

5. Criterio de fallo o éxito para las pruebas

Se dice que los requisitos asociados a su item de prueba han pasado cuando todos sus ítems de prueba y casos de prueba asociados han pasado también.

TEST CASES

1. Identificador del Test Case Specification (Caja Negra)

SW-TC 01.01-25

2. Test Items

La característica a probar es la AP06-01: Registro y gestión de usuarios.

3. Especificación de entradas

ID Entrada	Condición / Variable	Condición de Prueba
E01	Completar el formulario de registro correctamente (Correo electrónico)	Usuario ha ingresado sus datos de la forma esperada: Correo: user@test.com Contraseña: abcd1234
E02	Completar formulario con correo sin "@" (Correo electrónico)	Correo electrónico sin "@": Correo: usertest.com
E03	Escribir una contraseña con al menos 8 caracteres (Contraseña)	Contraseña con 8 caracteres como mínimo entre números y letras: Contraseña: abcd1234
E04	Intentar registrar un correo ya existente (Correo duplicado)	Correo ya registrado en el sistema
E05	Iniciar sesión con datos correctos (Login válido)	Usuario ha ingresado sus credenciales correctamente: email + password válidos
E06	Iniciar sesión con contraseña incorrecta (Login inválido)	Usuario ha ingresado una contraseña inválida: email válido + password erróneo

4. Especificación de salidas

ID Entrada	Resultado Esperado
E01	El sistema registra al usuario y muestra un mensaje confirmando el registro.
E02	El sistema muestra un mensaje advirtiendo que falta el símbolo "@".
E03	El sistema muestra un mensaje: "La contraseña debe tener al menos 8 caracteres".
E04	El sistema muestra un mensaje: "Usuario ya existe".
E05	El sistema permite el acceso y muestra la página principal.
E06	El sistema muestra un mensaje: "Credenciales inválidas".

5. Necesidades de entorno

5.1. Hardware

Cualquier PC de escritorio o portátil de gama media-baja en adelante.

5.2. Software

- Sistema operativo Windows 7 / 10 / 11.
- Cualquier navegador actualizado.
- Servidor backend activo (Node.js 18+ con base de datos MongoDB).

5.3. Otros

Base de datos inicial con registros de prueba conocidos.

6. Requisitos procedimentales especiales

- Verificar que la base de datos no contenga usuarios duplicados.
- Reiniciar el navegador para limpiar datos de sesión.

7. Dependencias entre casos

Ninguna.

1. Identificador del Test Case Specification (Caja Negra)

SW-TC 02.01-25

2. Test Items

La característica a probar es la AP06-02: Selección de productos del catálogo y carrito.

3. Especificación de entradas

ID Entrada	Condición / Variable	Condición de Prueba
E01	El usuario accede al catálogo (Autenticación)	Usuario ha iniciado sesión
E02	El usuario agrega un producto con stock disponible (Stock)	Producto con stock > 0
E03	El usuario aumenta la cantidad de un producto en el carrito (Cantidad)	Cantidad actual = 1 → nueva = 2
E04	El usuario elimina un producto del carrito (Remover producto)	Producto existente en el carrito

4. Especificación de salidas

ID Entrada	Resultado Esperado
E01	Se muestran los productos con su imagen, nombre y precio.
E02	El sistema añade el producto al carrito correctamente.
E03	El sistema actualiza la cantidad y recalcula el total correctamente.
E04	El sistema elimina el producto y actualiza el total.

5. Necesidades de entorno

5.1. Hardware

Cualquier PC de escritorio o portátil de gama media-baja en adelante.

5.2. Software

- Sistema operativo Windows 7 / 10 / 11.
- Cualquier navegador actualizado.
- Servidor backend activo (Node.js 18+ con base de datos MongoDB).

5.3. Otros

- Conjunto de productos cargados previamente.
- Cuenta de usuario registrada para realizar las pruebas.

6. Requisitos procedimentales especiales

- Restablecer el carrito después de cada caso de prueba.
- Comprobar que los totales coinciden con la suma de precios y cantidades.

7. Dependencias entre casos

Depende en parte de SW-TC 01.01-25 (Registro e inicio de sesión de usuarios), ya que se requiere un usuario válido para acceder al catálogo de productos y al carrito.

1. Identificador del Test Case Specification (Caja Negra)

SW-TC 03.01-25

2. Test Items

La característica a probar es la AP06-04: Registro y modificación de productos.

3. Especificación de entradas

ID Entrada	Condición / Variable	Condición de Prueba
E01	El usuario accede al apartado de gestión de productos	Credenciales válidas de administrador
E02	Registrar un nuevo producto	Usuario ha ingresado la información del producto de la forma esperada: ● Subir imagen ● Nombre del producto ● Descripción del producto ● Categoría ● Precio
E03	Registrar un producto con campo obligatorio vacío	Usuario no ha completado un campo obligatorio para registrar producto: Sin nombre o sin precio
E04	Ingresar el precio de un producto con un valor negativo	Usuario ha ingresado un valor negativo en el precio del producto: Precio: –5
E05	Eliminar un producto del catálogo	Usuario ha eliminado un producto correctamente.

4. Especificación de salidas

ID Entrada	Resultado Esperado
E01	Se muestran las vistas para registrar y eliminar un producto.
E02	El sistema registra el nuevo producto.
E03	El sistema muestra un mensaje: “Completa este campo”.
E04	El sistema no registra el producto.
E05	El sistema elimina el producto y lo retira del listado visible.

5. Necesidades de entorno

5.1. Hardware

Cualquier PC de escritorio o portátil de gama media-baja en adelante.

5.2. Software

- Sistema operativo Windows 7 / 10 / 11.
- Cualquier navegador actualizado.
- Servidor backend activo (Node.js 18+ con base de datos MongoDB).

5.3. Otros

Base de datos inicial con registros de productos para verificar y hacer modificaciones.

6. Requisitos procedimentales especiales

- Verificar que el cambio en la interfaz coincida con los datos guardados en la base de datos.
- Restablecer el catálogo de productos al estado inicial una vez concluidas las pruebas.

7. Dependencias entre casos

Depende en parte de SW-TC 01.01-25 (Registro e inicio de sesión de usuarios), ya que se requiere un usuario de tipo administrador para acceder al al apartado de gestión de productos.

1. Identificador del Test Case Specification (Caja Blanca)

SW-TC 04.01-25

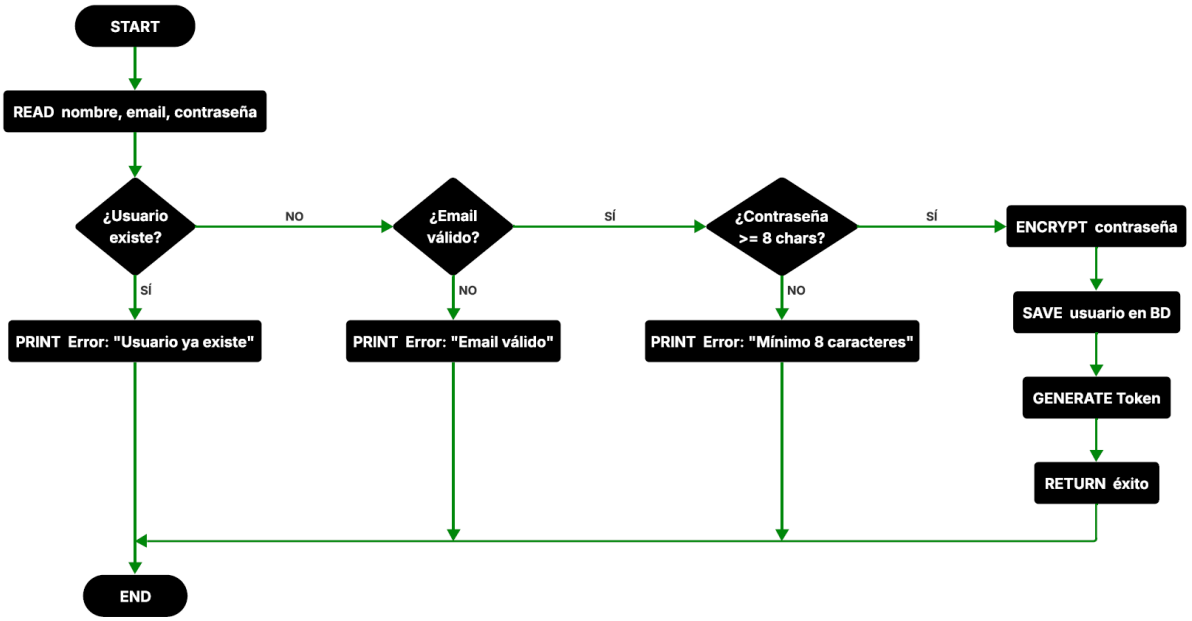
2. Test Items

La función a probar es: `registerUser` que se utiliza para el registro de usuarios.

3. Código de la función

```
const registerUser = async (req, res) => {
    const { nombre, email, contraseña } = req.body;
    try {
        // Verificar si el usuario ya existe (Decisión 1)
        const exists = await userModel.findOne({email});
        if(exists){
            return res.json({success:false, message:"Usuario ya existe"});
        }

        // Validar formato de email (Decisión 2)
        if(!validator.isEmail(email)){
            return res.json({success:false, message:"Por favor ingresar un email
válido"});
        }

        // Validar longitud de contraseña (Decisión 3)
        if(contraseña.length < 8){
            return res.json({success:false, message:"La contraseña debe tener al menos 8
caracteres"});
        }

        // Encriptar la contraseña
        const salt = await bcrypt.genSalt(10);
        const hashedPassword = await bcrypt.hash(contraseña, salt);

        // Crear y guardar nuevo usuario
        const newUser = new userModel({
            name: nombre,
            email: email,
            password: hashedPassword,
        });

        const user = await newUser.save();
        const token = createToken(user._id);
        res.json({success:true, token});

    } catch (error) {
        console.log("Error en registerUser:", error);
        res.json({ success: false, message: error.message });
    }
};
```

4. Diagrama de Flujo



5. Análisis de Complejidad Ciclomática

Número de decisiones: 3
Complejidad Ciclomática: 3 + 1 = 4

Caminos independientes:

- Camino 1: Usuario ya existe en BD
- Camino 2: Usuario nuevo → email inválido
- Camino 3: Usuario nuevo → email válido → contraseña < 8 caracteres
- Camino 4: Usuario nuevo → email válido → contraseña válida → registro exitoso

6. Casos de Prueba para Cobertura

6.1. Cobertura de Sentencias

Objetivo: Ejecutar todas las líneas de código al menos una vez
Casos de prueba mínimos requeridos: 1

Test ID	Nombre	Email	Contraseña	Camino	Sentencias Cubiertas
WB-04	Ana Torres	newuser@test.com	abcd1234	1→2→3→4→5→6	Todas (100%)

6.2. Cobertura de Decisiones

Objetivo: Ejecutar todos los branches (True/False) de cada decisión

Casos de prueba mínimos requeridos: 4

Test ID	Nombre	Email	Contraseña	Usuario Existe	Camino	Resultado Esperado
WB-01	Juan Pérez	existing@test.com	abcd1234	Sí	Usuario duplicado	{success:false, message:"Usuario ya existe"}
WB-02	María López	usertest.com	abcd1234	No	Email inválido	{success:false, message:"Por favor ingresar un email válido"}
WB-03	Carlos Ruiz	user@test.com	abc123	No	Contraseña corta	{success:false, message:"La contraseña debe tener al menos 8 caracteres"}
WB-04	Ana Torres	newuser@test.com	abcd1234	No	Registro exitoso	{success:true, token: "JWT_TOKEN"}

6.3. Matriz de Cobertura

Sentencia/Decisión	WB-01	WB-02	WB-03	WB-04
exists (True)	✓			
exists (False)		✓	✓	✓
!validator.isEmail() (True)		✓		
!validator.isEmail() (False)			✓	✓
contraseña.length < 8 (True)			✓	
contraseña.length < 8 (False)				✓
bcrypt.genSalt()				✓
bcrypt.hash()				✓
newUser.save()				✓
createToken()				✓

7. Especificación Detallada de Casos de Prueba

Test Case WB-01: Usuario ya existe en BD.

- **Entrada:**
  - nombre = "Juan Pérez"
  - email = "existing@test.com"
  - contraseña = "abcd1234"
- **Precondición:** Usuario con email "existing@test.com" ya existe en BD.
- **Resultado Esperado:** {success:false, message:"Usuario ya existe"}
- **Cobertura:** Decisión 1 (True), sentencias 1-5

Test Case WB-02: Email inválido.

- **Entrada:**
  - nombre = "María López"
  - email = "usertest.com"
  - contraseña = "abcd1234"
- **Precondición:** Usuario NO existe en BD.
- **Resultado Esperado:** {success:false, message:"Por favor ingresar un email válido"}
- **Cobertura:** Decisión 1 (False), Decisión 2 (True), sentencias 1-8

Test Case WB-03: Contraseña menor a 8 caracteres.

- **Entrada:**
  - nombre = "Carlos Ruiz"
  - email = "user@test.com"
  - contraseña = "abc123" (6 caracteres)
- **Precondición:** Usuario NO existe en BD.
- **Resultado Esperado:** {success:false, message:"La contraseña debe tener al menos 8 caracteres"}
- **Cobertura:** Decisión 1 (False), Decisión 2 (False), Decisión 3 (True), sentencias 1-11

Test Case WB-04: Registro exitoso.

- **Entrada:**
  - nombre = "Ana Torres"
  - email = "newuser@test.com"
  - contraseña = "abcd1234"
- **Precondición:** Usuario NO existe en BD.
- **Resultado Esperado:** {success:true, token: "JWT_TOKEN"}
- **Cobertura:** Todas las decisiones (False, False, False), todas las sentencias (100%)

8. Necesidades de entorno

8.1. Hardware

Cualquier PC de escritorio o portátil de gama media-baja en adelante.

8.2. Software

- Sistema operativo Windows 7 / 10 / 11.
- Cualquier navegador actualizado.
- Servidor backend activo (Node.js 18+ con base de datos MongoDB).

8.3. Herramientas de Cobertura

- Istanbul/nyc para JavaScript/Node.js.
- Jest con --coverage flag.
- SonarQube para análisis de cobertura.

9. Requisitos procedimentales especiales

- Acceso al código fuente de la función.
- Instrumentación del código para medir cobertura.
- Base de datos de prueba con datos conocidos.
- Capacidad de resetear BD entre pruebas.

Identificador del Test Case Specification (Caja Blanca)

SW-TC 05.01-25

Test Items

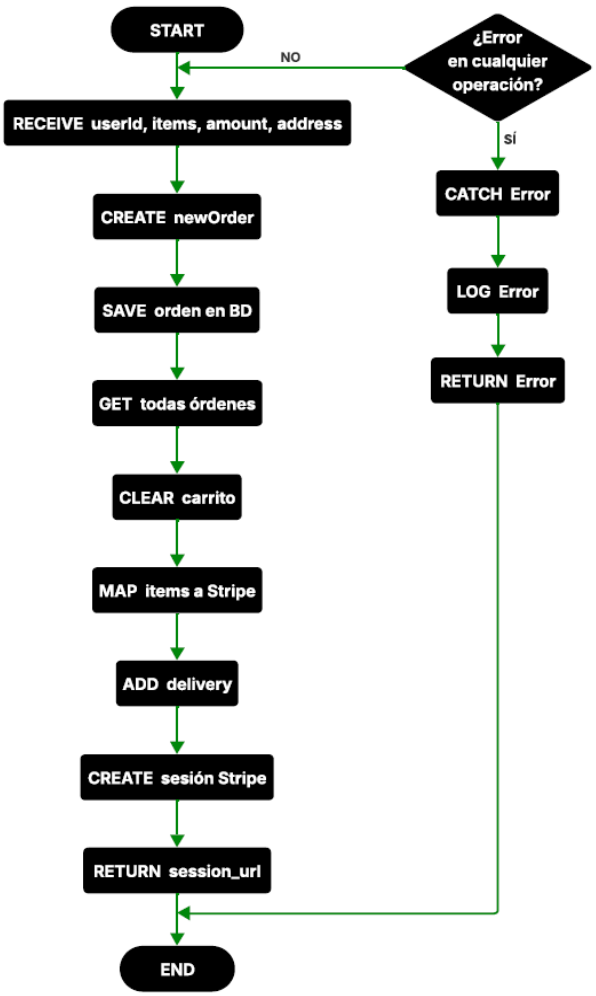
La función a probar es: `placeOrder` que se utiliza para crear una orden de compra.

Código de la función

```
const placeOrder = async (req, res) => {
    const frontendUrl = "http://localhost:5173";

    try {
        // Crear nueva orden con datos del usuario (Bloque 1)
        const newOrder = new orderModel({
            userId: req.body.userId,
            items: req.body.items,
            amount: req.body.amount,
            address: req.body.address,
        });

        // Guardar la orden en la base de datos (Bloque 2)
        await newOrder.save();
        const orders = await orderModel.find({});
        console.log("Órdenes después de insertar:", orders);

        // Limpiar carrito del usuario (Bloque 3)
        await userModel.findByIdAndUpdate(req.body.userId, {cartData:{}});

        // Mapear items a formato Stripe (Bloque 4)
        const line_items = req.body.items.map((item)=>({
            price_data:{
                currency: "pen",
                product_data:{
                    name: item.name,
                },
                unit_amount: item.price * 100,
            },
            quantity: item.quantity
        }));

        // Agregar cargo por delivery (Bloque 5)
        line_items.push({
            price_data:{
                currency: "pen",
                product_data:{
                    name: "Delivery Charges",
                },
                unit_amount: 2*100,
            },
            quantity: 1,
```

```
        });

        console.log("usa la siguiente tarjeta: 4000006040000068");

        // Crear sesión de pago con Stripe (Bloque 6)
        const session = await stripe.checkout.sessions.create({
            line_items: line_items,
            mode: 'payment',
            success_url: `${frontendUrl}/verify?success=true&orderId=${newOrder._id}`,
            cancel_url: `${frontendUrl}/verify?success=false&orderId=${newOrder._id}`
        });

        // Respuesta exitosa (Bloque 7)
        res.json({success:true, session_url: session.url});

    } catch (error) {
        // Manejo de errores (Bloque 8)
        console.log("Error al crear la orden: ", error);
        res.json({success:false, message: "Error al crear la orden"});
    }
};
```

Diagrama de Flujo



Análisis de Complejidad Ciclomática

Número de decisiones: 1
Complejidad Ciclomática: 1 + 1 = 2

Caminos independientes:
- Camino 1: Flujo exitoso → todas las operaciones se completan correctamente
- Camino 2: Flujo con error → alguna operación falla y se captura en catch

Casos de Prueba para Cobertura

6.1. Cobertura de Sentencias

Objetivo: Ejecutar todas las líneas de código al menos una vez
Casos de prueba mínimos requeridos: 2

Test ID	UserID	Items	Amount	Address	Camino	Sentencias Cubiertas
WB-01	validId	[{…}]	50.00	{...}	Exitoso	Bloque try completo
WB-02	invalidId	[{…}]	50.00	{...}	Error	Bloque catch

6.2. Cobertura de Decisiones

Objetivo: Ejecutar el bloque try exitoso y el bloque catch con error
Casos de prueba mínimos requeridos: 2

Test ID	Descripción	UserID	Items	Amount	Address	Resultado Esperado
WB-01	Orden exitosa con productos válidos	"60f7b3b3b3b3b3b3b3b3b3b3"	[{"name":"Producto A","price":25,"quantity":2}]	52.00	{"street":"Av. Lima 123","city":"Lima"}	{success:true, session_url:"stripe_url"}
WB-02	Error por userId inválido	"invalid_id"	[{"name":"Producto A","price":25,"quantity":2}]	52.00	{"street":"Av. Lima 123","city":"Lima"}	{success:false, message:"Error al crear la orden"}
WB-03	Error por fallo en BD	"60f7b3b3b3b3b3b3b3b3b3b3"	[{"name":"Producto A","price":25,"quantity":2}]	52.00	{"street":"Av. Lima 123","city":"Lima"}	{success:false, message:"Error al crear la orden"}

WB-04	Error por fallo en Stripe	"60f7b3b3b3b3b3b3b3b3b3b3"	[{"name":"Producto A","price":25,"quantity":2}]	52.00	{"street":"Av. Lima 123","city":"Lima"}	{success:false, message:"Error al crear la orden"}

6.3. Matriz de Cobertura

Sentencia/Decisión	WB-01	WB-02	WB-03	WB-04
Crear newOrder	✓	✓	✓	✓
newOrder.save() (éxito)	✓			✓
newOrder.save() (fallo)		✓	✓	
orderModel.find()	✓			✓
userModel.findByIdAndUpdate()	✓			✓
items.map()	✓			✓
line_items.push()	✓			✓
stripe.checkout.sessions.create() (éxito)	✓			
stripe.checkout.sessions.create() (fallo)				✓
res.json() success	✓			
catch block		✓	✓	✓
console.log() error		✓	✓	✓
res.json() error		✓	✓	✓

Especificación Detallada de Casos de Prueba

Test Case WB-01: Creación exitosa de orden

- **Entrada:**
  - userId = "60f7b3b3b3b3b3b3b3b3b3b3" (válido)
  - items = [{name: "Laptop HP", price: 1500, quantity: 1}, {name: "Mouse Logitech", price: 50, quantity: 2}]
  - amount = 1602.00 (1500 + 50*2 + 2 delivery)
  - address = { street: "Av. Arequipa 456", city: "Lima", district: "Miraflores", phone: "987654321" }

- **Precondiciones:**
  - Usuario existe en BD
  - Productos tienen stock
  - Conexión con Stripe activa
- **Resultado Esperado:**
  `{ success: true, session_url: "https://checkout.stripe.com/c/pay/cs_test_xxx"}`
- **Postcondiciones:**
  - Orden guardada en BD
  - Carrito del usuario vaciado
  - Sesión de Stripe creada
- **Cobertura:** Todas las sentencias del bloque try (100%)

Test Case WB-02: Error por userId inválido

- **Entrada:**
  - userId = "invalid_user_id"
  - items = [{name: "Producto Test", price: 25, quantity: 2}]
  - amount = 52.00
  - address = {street: "Test 123", city: "Lima"}
- **Precondiciones:**
  - userId no existe o tiene formato inválido
- **Resultado Esperado:**
  `{ success: false, message: "Error al crear la orden"}`
- **Cobertura:** Bloque catch ejecutado

Test Case WB-03: Error al guardar en Base de Datos

- **Entrada:**
  - userId = "60f7b3b3b3b3b3b3b3b3b3b3"
  - items = [{name: "Producto Test", price: 25, quantity: 2}]
  - amount = 52.00
  - address = {street: "Test 123", city: "Lima"}
- **Precondiciones:**
  - Base de datos no disponible o error de conexión
  - Simular fallo en `newOrder.save()`
- **Resultado Esperado:**
  `{ success: false, message: "Error al crear la orden"}`
- **Cobertura:** Bloque catch ejecutado

Test Case WB-04: Error al crear sesión de Stripe

- **Entrada:**
  - userId = "60f7b3b3b3b3b3b3b3b3b3b3"
  - items = [{name: "Producto Test", price: 25, quantity: 2}]
  - amount = 52.00
  - address = {street: "Test 123", city: "Lima"}
- **Precondiciones:**
  - API de Stripe no disponible o credenciales inválidas

- Simular fallo en `stripe.checkout.sessions.create()`

- **Resultado Esperado:**
  `{ success: false, message: "Error al crear la orden"}`
- **Cobertura:** Bloque try hasta el punto de fallo + bloque catch

Necesidades de entorno

8.1. Hardware

Cualquier PC de escritorio o portátil de gama media-baja en adelante.

8.2. Software

- Sistema operativo Windows 7 / 10 / 11.
- Cualquier navegador actualizado.
- Servidor backend activo (Node.js 18+ con base de datos MongoDB).
- Cuenta de Stripe (modo test).

8.3. Herramientas de Cobertura

- Istanbul/nyc para JavaScript/Node.js.
- Jest con --coverage flag.
- Postman/Insomnia para pruebas de API
- SonarQube para análisis de cobertura.

8.4. Variables de Entorno

- `STRIPE_SECRET_KEY` configurada
- `FRONTEND_URL` = "http://localhost:5173"

Requisitos procedimentales especiales

- Usuario válido registrado en BD con cartData poblado.
- Productos válidos en catálogo.
- Verificar que la orden se guarde correctamente en BD antes de crear sesión Stripe.
- Confirmar que el carrito se vacíe después de crear la orden.
- Limpiar órdenes de prueba de la BD después de cada test

1. Identificador del Test Case Specification (Caja Blanca)

SW-TC 06.01-25

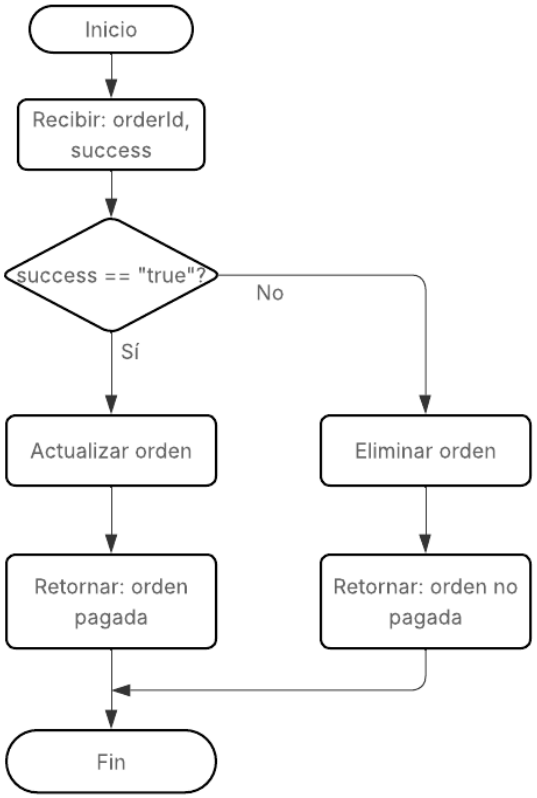
2. Test Items

La función a probar es: `verifyOrder` encargada de verificar si una orden fue pagada correctamente para establecer su estado final (pagado / no pagado).

3. Código de la función

```
const verifyOrder = async (req, res) => {
    const {orderId,success}=req.body;
    try{
        if(success=="true"){
            await orderModel.findByIdAndUpdate(orderId, {payment:true});
            res.json({success:true, message:"Paid"})
        }
        else{
            await orderModel.findByIdAndDelete(orderId);
            res.json({success:false, message:"Not paid"});
        }
    }catch(error){
        console.log(error);
        res.json({success:false, message:"Error al verificar la orden"})
    }
}
```

4. Diagrama de flujo



5. Análisis de complejidad ciclomática

Número de decisiones: 1
Complejidad Ciclomática: 1 + 1 = 2
Caminos independientes:

- Camino 1: success == “true” → Actualizar Pago → Retorna “Paid”
- Camino 2: success != “true” → Eliminar Orden → Retorna “Not Paid”

6. Casos de prueba para cobertura

6.1 Cobertura de Sentencias

Objetivo: Ejecutar todas las líneas de código al menos una vez
Con 2 casos se cubren todas las líneas

Test ID	success	orderId	Camino	Sentencias Cubiertas
WB-01	“true”	válido	Camino 1	Rama True
WB-02	“false”	válido	Camino 2	Rama False

6.2 Cobertura de Decisiones

Objetivo: Ejecutar todos los branches (True/False) de cada decisión

Test ID	Condicion	Valor de entrada	Resultado booleano	Rama que se ejecuta	Resultado Esperado
WB-01	success == “true”	“true”	TRUE	Ruta de actualización de pago	{success:true, message:"Orden Pagada"}
WB-02	success == “true”	“false”	FALSE	Ruta de eliminación de orden	{success:false, message:"Orden no pagada"}

6.3 Matriz de cobertura

Sentencia/Decisión	WB-01	WB-02
success == "true" (True)	✓	
success == "true" (False)		✓
findByIdAndUpdate()	✓	
findByIdAndDelete()		✓

res.json() Paid	✓	
res.json() Not paid		✓

7. Especificación detallada de casos de prueba

Test Case WB-01: Orden pagada correctamente.

- **Entrada:**
  - orderId = 6745ab99123abc55de890123
  - success = "true”
- **Precondición:** La orden existe en la base de datos y la BD está operativa.
- **Resultado Esperado:** {success:true, message:"Paid"}
- **Cobertura:** Decisión 1 (True)

Test Case WB-02: Orden no pagada.

- **Entrada:**
  - orderId = "6745ab99123abc55de890124"
  - success = "false”
- **Precondición:** La orden existe en la base de datos.
- **Resultado Esperado:** {success:false, message:"Not Paid"}
- **Cobertura:** Decisión 1 (False)

8. Necesidades de entorno

8.1 Hardware

Cualquier PC de escritorio o portátil de gama media-baja en adelante.

8.2 Software

- Sistema operativo Windows 7 / 10 / 11.
- Cualquier navegador actualizado.
- Servidor backend activo (Node.js 18+ con base de datos MongoDB).

8.3 Herramientas de Cobertura

- Istanbul/nyc para JavaScript/Node.js.
- Jest con --coverage flag.
- SonarQube para análisis de cobertura.

9. Requisitos procedimentales especiales

- BD debe contener órdenes definidas para la prueba
- Acceso al código fuente de la función.
- Se debe verificar visualmente la actualización/eliminación del registro.
- Capacidad de resetear BD entre pruebas.

1. Identificador del Test Case Specification (Caja Blanca)

SW-TC-07.01-25

2. Test Items

Función a probar:
loginUser (backend/controllers/userController.js)

3. Código de la función

```
const loginUser = async (req, res) => {
  const { email, contraseña } = req.body;
  try {
    // 1. Buscar usuario por email
    const user = await userModel.findOne({ email });
    if (!user) {
      return res.json({ success: false, message: "Credenciales inválidas" });
    }
    // 2. Comparar contraseña enviada y la almacenada
    const match = await bcrypt.compare(contraseña, user.password);
    if (!match) {
      return res.json({ success: false, message: "Credenciales inválidas" });
    }
    // 3. Si todo correcto, crear token de sesión
    const token = createToken(user._id);
    res.json({ success: true, token });
  } catch (error) {
    res.json({ success: false, message: error.message });
  }
}
```

4. Análisis de Complejidad Ciclomática

- Decisión 1: ¿Usuario existe?
- Decisión 2: ¿Contraseña coincide?
- Ruta de try/catch de error

Nº Decisiones: 2
Complejidad Ciclomática: 3
(Caminos independientes = nº decisiones + 1)

5. Caminos independientes

a. Usuario no existe (branch “user == null”)
b. Usuario existe, contraseña incorrecta
c. Usuario existe, contraseña correcta (login exitoso)
d. Ocurre un error de sistema (catch)

6. Casos de prueba para cobertura de sentencias y decisiones

Test ID	email	contraseña	Precondición	Camino	Resultados esperado
WB-06-01	noexiste@mail.com	abcd1234	Usuario NO existe en BD	Decisión 1 True	success: false, message: "Credenciales inválidas"
WB-06-02	user@mail.com	erron123	Usuario existe, pass incorrecta	Dec 1 False, Dec 2 True	success: false, message: "Credenciales inválidas"
WB-06-03	user@mail.com	abcd1234	Usuario existe, pass correcta	Dec 1 False, Dec 2 False	success: true, token: JWTTOKEN
WB-06-04	<cualquiera>	<cualquiera>	Error forzado (fallo BD)	throw / catch block	success: false, message: error simulado

7. Matriz de Cobertura

Sentencia / Decisión	WB-06-01	WB-06-02	WB-06-03	WB-06-04
user == null	✓			
!match (contraseñas)		✓		
match / token			✓	
catch (error)				✓

8. Casos detallados (con datos simulados)

- WB-06-01:
  - Entrada: email=noexiste@mail.com, contraseña=abcd1234
  - Precondición: No existe ningún usuario con ese email
  - Resultado Esperado: { success: false, message: "Credenciales inválidas" }

- WB-06-02:
  - Entrada: email=cliente@correo.com, contraseña=passIncorrecta
  - Precondición: Usuario existe, contraseña en BD = passCorrectA1
  - Resultado Esperado: { success: false, message: "Credenciales inválidas" }

- WB-06-03:
  - Entrada: email=user@mail.com, contraseña=passCorrectA1
  - Precondición: Usuario existe con ese email y contraseña
  - Resultado Esperado: { success: true, token: "eyJhbGciOi..." }

- WB-06-04:
    - Entrada: cualquier email y contraseña
    - Precondición: Se simula un error en la conexión a la BD o error de código
    - Resultado Esperado: { success: false, message: "Error simulado en BD" }

9. Necesidades especiales

- BD con al menos un usuario válido para pruebas
- Capacidad de limpiar/inyectar datos
- Herramienta de cobertura (Jest, Istanbul)
- Node.js >= 18, MongoDB de prueba

TEST PROCEDURES

1. Identificador del Test Procedure

SW-TProc 01.01-25

2. Propósito

Verificación del proceso completo de selección de productos, gestión del carrito y proceso de checkout del sistema.

3. Requisitos especiales

- Credenciales de pruebas tanto en permisos de usuario como administrador
- Datos de prueba pre-cargados en base a productos en stock

4. Pasos de procedimientos de pruebas

4.1. Log

Generar un archivo de la ejecución de este Test Procedure junto al documento del Test Case que le pertenece.

4.2. Set-Up

Verificación del proceso completo de selección de productos, gestión del carrito y proceso de checkout del sistema.

(1) Abrir una terminal desde la carpeta frontend y usar npm para iniciar un localhost
(2) Hacer (1) con las carpetas admin y backend

4.3. Proceed

Ejecutar los test case en el siguiente orden:

(1) SW-TC 01.01-25 (Registro / Login)
(2) SW-TC 01.02-25 (Carrito)

4.4. Measure

Métrica	Descripción	Unidad de Medida
Tiempo de Ejecución	Tiempo total transcurrido desde el inicio hasta la finalización de un *Test Case*.	Minutos / Segundos
Defectos Encontrados	Número de defectos (bugs) detectados durante la ejecución.	Cantidad (N°)
Severidad de Defectos	Nivel de impacto del defecto (e.g., Crítico, Mayor, Menor, Sugerencia).	Clasificación (Texto)

4.5. Shut down

(1) Cerrar la sesión
(2) Cerrar las terminales

4.6. Wrap-up

Documentar los resultados de las pruebas, tales como anomalías durante la ejecución

4.7. Contingencias

Ante problemas de ejecución revisar que los archivos .env existan

1. Identificador del Test Procedure

SW-TProc 02.01-25

2. Propósito

Ejecución de los casos de prueba derivados del test case SW-TC 04.01-25 para verificar la lógica interna del IP-02 relacionado con el registro de usuario

3. Casos de prueba a ejecutar

Del SW-TC 04.01-25: SW-TC-WB-01 al SW-TC-WB-04

4. Requisitos especiales

- Node.js v18+.
- Dependencias de desarrollo instaladas (npm install).

5. Pasos de procedimientos de pruebas

5.1. Log

Generar un archivo de la ejecución de este Test Procedure junto al documento del Test Case que le pertenece.

5.2. Set-Up

Verificación del funcionamiento adecuado del controlador registerUser. Con respuestas adecuadas del backend al recibir la solicitud.

(1) Generar en la colección users un usuario cualquiera con el correo existing@test.com
(2) Iniciar el servidor local de la aplicación

5.3. Proceed

Orden	Acción	Referencia al Test Case	Verificación Inmediata
1	Inyectar datos de usuario existente definidos en SW-TC-04.01.	Ejecutar SW-TC-04.01	Confirmar mensaje de error "Usuario ya existe".
2	Sin reiniciar, inyectar datos de email inválido definidos en SW-TC-04.02.	Ejecutar SW-TC-04.02	Confirmar mensaje de validación de email.
3	Sin reiniciar, inyectar datos de password corto definidos en SW-TC-04.03.	Ejecutar SW-TC-04.03	Confirmar mensaje de validación de longitud.

4	Inyectar datos válidos definidos en SW-TC-04.04.	Ejecutar SW-TC-04.04	Confirmar Token recibido y HTTP 200.

5.4. Measure

Métrica	Descripción	Unidad de Medida
Cobertura de sentencias	Porcentaje de líneas de código ejecutadas respecto al total de la función registerUser que obtuvieron la respuesta esperada	Porcentaje
Cobertura de decisiones	Porcentaje de ramas de decisión (éxito/fallo en try-catch) que obtuvieron la respuesta esperada	Porcentaje

5.5. Shut down

(1) Cerrar la sesión
(2) Cerrar las terminales

5.6. Wrap-up

Revisar que el reporte de coberturas de sentencias y condiciones generado sea de al menos 70%

5.7. Contingencias

Si la base de datos no responde reconfigurar la conexión en el config o revisar el estado de MongoDB. Luego repetir desde setup

1. Identificador del Test Procedure

SW-TProc 03.01-25

2. Propósito

Ejecución de los casos de prueba derivados del test case SW-TC 05.01-25 para verificar la lógica interna del IP-05 relacionado con la solicitud de pedidos

3. Casos de prueba a ejecutar

Del SW-TC 05.01-25: SW-TC-WB-01 al SW-TC-WB-04

4. Requisitos especiales

- Node.js v18+.
- Dependencias de desarrollo instaladas (npm install).
- Variables de entorno configuradas (Stripe)

5. Pasos de procedimientos de pruebas

5.1 Log

Generar un archivo de la ejecución de este Test Procedure junto al documento del Test Case que le pertenece.

5.2 Set-Up

Verificación del funcionamiento adecuado del controlador registerUser. Con respuestas adecuadas del backend al recibir la solicitud.

(3) Revisar que exista el usuario “60f7b3b3…b3” en la base de datos
(4) Iniciar el servidor local de la aplicación

5.3 Proceed

Orden	Acción	Referencia al Test Case	Verificación Inmediata
1	Inyectar datos de usuario existente definidos en SW-TC-05.01.	Ejecutar SW-TC-05.01	Confirmar respuesta {success: true}.
2	Sin reiniciar, inyectar datos de email inválido definidos en SW-TC-05.02.	Ejecutar SW-TC-05.02	Confirmar mensaje “Error al crear orden”

3	Sin reiniciar, inyectar datos de password corto definidos en SW-TC-05.03.	Ejecutar SW-TC-05.03	Confirmar la obtención del error en el bloque catch
4	Inyectar datos válidos definidos en SW-TC-05.04.	Ejecutar SW-TC-05.04	Confirmar la obtención del error del bloque catch

5.4 Measure

Métrica	Descripción	Unidad de Medida
Cobertura de sentencias	Porcentaje de líneas de código ejecutadas respecto al total de la función placeOrder que obtuvieron la respuesta esperada	Porcentaje
Cobertura de decisiones	Porcentaje de ramas de decisión (éxito/fallo en try-catch) que obtuvieron la respuesta esperada	Porcentaje

5.5 Shut down

(3) Cerrar la sesión
(4) Cerrar las terminales

5.6 Wrap-up

Revisar que el reporte de coberturas de sentencias y condiciones generado sea de al menos 70%

5.7 Contingencias

Si la base de datos no responde reconfigurar la conexión en el config o revisar el estado de MongoDB. Luego repetir desde setup
Si la API de Stripe no responde revisar la clave en el archivo de entorno .env

1. Identificador del Test Procedure

SW-TProc 04.01-25

2. Propósito

Ejecución de los casos de prueba derivados del test case SW-TC 07.01-25 para verificar la lógica interna del IP-01 relacionado con el controlador de login

3. Casos de prueba a ejecutar

Del SW-TC 07.01-25: SW-TC-WB-01 al SW-TC-WB-04

4. Requisitos especiales

- Node.js v18+.
- Dependencias de desarrollo instaladas (npm install).
- Variables de entorno configuradas (Stripe)

5. Pasos de procedimientos de pruebas

5.1 Log

Generar un archivo de la ejecución de este Test Procedure junto al documento del Test Case que le pertenece.

5.2 Set-Up

Verificación del funcionamiento adecuado del controlador registerUser. Con respuestas adecuadas del backend al recibir la solicitud.

(5) Revisar que exista el usuario con email: user@mail.com existe en la base de datos
(6) Iniciar el servidor local de la aplicación

5.3 Proceed

Orden	Acción	Referencia al Test Case	Verificación Inmediata
1	Inyectar datos de usuario existente definidos en SW-TC-07.01.	Ejecutar SW-TC-07.01	Confirmar respuesta {success: false}.
2	Sin reiniciar, inyectar datos de email inválido definidos en SW-TC-07.02.	Ejecutar SW-TC-07.02	Confirmar respuesta {success: false}

3	Sin reiniciar, inyectar datos de password corto definidos en SW-TC-07.03.	Ejecutar SW-TC-07.03	Confirmar respuesta {success: true}
4	Inyectar datos válidos definidos en SW-TC-07.04	Ejecutar SW-TC-07.04	Confirmar respuesta {success: false} y el error generado por la conexión a la BD

5.4 Measure

Métrica	Descripción	Unidad de Medida
Cobertura de sentencias	Porcentaje de líneas de código ejecutadas respecto al total de la función userLogin() y que obtuvieron la respuesta esperada	Porcentaje
Cobertura de decisiones	Porcentaje de ramas de decisión que obtuvieron la respuesta esperada	Porcentaje

5.5 Shut down

(5) Cerrar la sesión
(6) Cerrar las terminales

5.6 Wrap-up

Revisar que el reporte de coberturas de sentencias y condiciones generado sea de al menos 70%

5.7 Contingencias

Si la base de datos no responde reconfigurar la conexión en el config o revisar el estado de MongoDB.